

## MANUAL26. УРОК 14. SMOKE И SANITY: КАК НЕ ТРАТИТЬ ТЕСТОВОЕ ОКНО ВПУСТУЮ.

### **Почему регрессию опасно запускать сразу после получения билда?**

Потому что если базовая функциональность уже повреждена, глубокая проверка почти всегда превращается в потерю времени. Команда собирает длинный список симптомов, но не получает ясного ответа: билд вообще пригоден для дальнейшей работы или его нужно сразу возвращать в доработку.

---

### **Что такое smoke с точки зрения цели?**



Smoke

это входной тест на  
жизнеспособность.

Его задача не «доказать  
качество», а быстро проверить,  
что продукт в принципе способен  
пройти ключевой  
пользовательский путь без  
критического сбоя.

---

## Что такое sanity с точки зрения цели?

### Sanity

это узкая проверка конкретной  
правки.

После фикса вы не  
перепроверяете весь продукт, а  
отвечаете на вопрос:  
«измененная зона теперь ведет  
себя адекватно в базовых  
сценариях?»

---

## Почему важно не путать smoke и sanity?

Потому что они отвечают на  
разные риски:

1. smoke — риск «билд не готов к тестированию»;
2. sanity — риск «конкретный фикс некачественный».

Если смешать их, команда теряет скорость и ясность решения.

---

## Как выглядит минимально полезный smoke-набор?

Обычно он включает:

1. запуск приложения/клиента;
2. авторизацию;
3. вход в ключевой раздел;
4. выполнение базового сценария;
5. корректный выход/завершение.

Набор должен проходиться быстро, иначе это уже не входной контроль.

---

## Как выглядит минимально полезный sanity-набор?

Он строится от конкретного изменения:

1. проверка исправленного сценария;
2. проверка ближайших смежных переходов;
3. короткая проверка на очевидные побочные эффекты.

Sanity не должен превращаться в мини-регрессию «на всякий случай».

---

## В каком порядке запускать проверки?

Рабочий маршрут обычно такой:

1. сначала smoke;
2. потом sanity по ключевым фиксам;
3. затем регрессия нужной глубины.

Если smoke падает по критичной зоне, продолжение проверки без согласования обычно бессмысленно.

---

## Что такое gate-критерии и почему они критичны?

### Gate-критерии

это заранее согласованные  
правила "идем дальше/стоп".

Без них команда начинает спорить уже в процессе теста: что считать блокером, какие дефекты допустимы временно, кто принимает финальное решение.

---

## Как формулировать gate-критерии на практике?

Полезно зафиксировать:

1. какие дефекты автоматически блокируют переход;
2. какие допускаются как известный риск;
3. кто принимает решение по спорным случаям;
4. где фиксируется итог (задача, чат, релизный отчет).

Когда критерии прозрачны, команда движется намного быстрее.

---

## Как это работает на кейсе Apex Legends?

Команда получает ночной билд. QA запускает smoke: клиент, логин, лобби, старт матча. После

успешного входного шага запускается sanity на фикс экрана бонусов события. Sanity падает на переходе «бонусы → профиль». Решение принимается локально: вернуть фикс в доработку, а не размазывать проблему по полной регрессии.

---

## Почему «длинный smoke» — это антипаттерн?

Потому что длинный smoke теряет смысл: вместо быстрого go/no-go получается неполная регрессия без приоритетов. В итоге ни скорости, ни ясного решения.

---

## Почему «размытый sanity» — тоже антипаттерн?

Когда команда не знает, что именно менялось, sanity превращается в хаотичную проверку «еще пары экранов». Такой подход плохо ловит реальную проблему и часто пропускает регрессионные эффекты в нужной зоне.

---

## Какие ошибки чаще всего допускают начинающие QA?

Чаще всего:

1. пропускают smoke и сразу идут в глубину;
  2. не фиксируют pass/fail явно;
  3. не описывают дальнейший шаг после fail;
  4. запускают sanity без списка изменений;
  5. принимают решение устно без записи;
  6. начинают регрессию на билде с неразрешенными блокерами.
- 

## Как проверить качество входного контроля перед началом окна?

Быстрый чек:

1. актуален ли smoke-набор для текущей версии;
  2. понятны ли зоны sanity;
  3. согласованы ли gate-критерии;
  4. назначен ли ответственный за финальный go/no-go;
  5. зафиксирован ли сценарий действий при fail.
-

## Как понять, что входной контроль сработал правильно?

Если команда быстро получила однозначный ответ: продолжаем проверку или возвращаем билд. Если после часа тестирования этот ответ так и не появился, значит входной этап был построен неправильно.

## Почему smoke/sanity — это управленческий инструмент, а не «техническая рутина»?

Потому что они напрямую влияют на распределение времени команды в критичное окно. Хороший входной контроль экономит десятки человеко-часов и снижает вероятность релизных ошибок, вызванных спешкой.

---

## Какой главный вывод урока?

Smoke и sanity нужны, чтобы в начале тестового окна быстро и обоснованно решить, куда направить усилия команды. Это базовая дисциплина, которая защищает и качество, и сроки.



## Какой мостик к следующему уроку?

После того как вы научились входному контролю сборки, следующий шаг — проверить продукт в реальных средах использования. В уроке 15 разберем кросс-браузерное и кросс-платформенное покрытие.

## Почему smoke важен даже в очень стабильных командах?

Потому что стабильность не отменяет риск неожиданных сбоев на этапе сборки, конфигурации или интеграции. Короткий smoke защищает команду от ситуации, когда несколько часов уходят на регрессию билда, который изначально не проходил базовые условия.

---

## Как определить оптимальную длину smoke-набора?

Набор считается оптимальным, если:

1. проходит быстро;

2. покрывает ключевые пользовательские цели;
3. дает однозначный go/no-go сигнал.

Если набор растет от релиза к релизу и перестает быть быстрым, его нужно чистить и возвращать к исходной задаче.

---

## Как избежать «размывания» sanity-проверки?

Перед запуском sanity полезно зафиксировать три вещи: какая зона менялась, какой сценарий нужно подтвердить и какой результат считается fail. Без этой рамки sanity быстро превращается в хаотичный мини-прогон и теряет ценность.

---

## Что делать, если smoke проходит, а sanity падает?

Это хороший диагностический сигнал: базовая сборка жизнеспособна, но конкретный фикс не готов. В таком случае команда может локализованно вернуть задачу в доработку, не останавливая весь цикл качества.

## Почему важно сохранять результаты входного контроля в одном формате?

Единый формат (pass/fail + причина + следующий шаг) снижает количество недопониманий и помогает анализировать качество процесса ретроспективно. Через несколько релизов это дает полезную статистику: какие зоны чаще ломают вход в регрессию.

---

## Какие команды чаще всего теряют время на входном этапе?

Те, где:

1. нет явных gate-критериев;
2. решение принимается устно и не фиксируется;
3. smoke и sanity не разделены;
4. никто не отвечает за финальный go/no-go.

Даже сильная техническая команда при таком процессе будет терять ритм.

---

## Какой дополнительный практический результат дает урок?

Вы начинаете использовать smoke и sanity как инструмент управления временем: быстро отделяете «сборка не готова» от «локальный фикс не готов», а значит принимаете решения быстрее и точнее.

---

## Как корректировать smoke-набор по мере роста продукта?

Полезно проводить ревизию после крупных релизов: удалять устаревшие шаги, добавлять новые критичные точки и проверять общее время прогона. Smoke живет вместе с продуктом. Если не пересматривать набор, он постепенно превращается в тяжелый регресс и теряет свою основную функцию — быстрый входной сигнал.

---

## Когда sanity стоит расширять, а когда — наоборот сокращать?

Расширять стоит, если изменение затрагивает общий компонент, от которого зависят несколько ключевых сценариев. Сокращать стоит, если фикс локальный и четко ограничен одной функциональной зоной. В обоих случаях цель одна: получить достоверный ответ о качестве изменения без лишних действий.

---

## Как команда может визуализировать результаты входного контроля?

Удобен короткий статус-блок в релизном чате или задаче:

1. smoke: pass/fail;
2. sanity по каждому критичному фиксу;
3. блокирующие проблемы;
4. решение по переходу к регрессии.

Такой формат помогает всем ролям быстро понять текущее состояние сборки.

---

## Почему важно указывать причину fail, а не только сам факт?

Потому что «smoke fail» без контекста не позволяет принять решение. Причина fail задает маршрут: вернуть билд, продолжить ограниченную проверку или выделить отдельный triage. Чем точнее причина, тем меньше лишних обсуждений.

---

## **Какие признаки показывают, что входной контроль работает хорошо?**

Если команда регулярно получает go/no-go решение в предсказуемое время, а количество «пустых» часов на неготовых билдах снижается, значит smoke/sanity встроены правильно и реально экономят тестовое окно.

---

## **Как использовать результаты smoke/sanity для улучшения релизного процесса?**

После нескольких циклов полезно анализировать, какие проверки чаще всего дают fail и

на каких этапах возникают проблемы. Это помогает не только улучшать тестовый набор, но и обнаруживать системные слабости сборочного процесса.

---

## **Почему важно заранее договориться о канале коммуникации входного контроля?**

Когда команда знает, где именно публикуется итог smoke/sanity, меньше риска пропустить критичную информацию. Единый канал и единый формат сообщения ускоряют принятие решения по всей цепочке ролей.

---

## **Что нужно закрепить перед практическим разбором билда?**

Проверьте, что для вас очевидно различие: smoke отвечает за готовность сборки, sanity — за качество конкретного изменения. Если это различие не удерживается, входной этап быстро теряет скорость и предсказуемость.

---

## Как выглядит хороший результат урока в реальной команде?

Когда после получения билда решение «идем дальше/останавливаемся» принимается быстро, спокойно и на основе заранее согласованных критериев, а не на основе импровизации в чате.

---

## Почему после каждого релиза стоит кратко разбирать входной этап?

Короткий разбор показывает, где команда потеряла время: на неактуальном smoke-наборе, на размытом sanity или на позднем согласовании критериев. Даже 10–15 минут такого анализа после релиза помогают стабильно улучшать входной контроль и сокращать количество «пустых» проверок в следующем цикле.

---

## Глоссарий к занятию



1. **Smoke-тестирование** — короткая проверка базовой работоспособности сборки перед глубокой проверкой.
2. **Sanity-тестирование** — точечная проверка конкретного изменения после фикса.
3. **Входной контроль** — этап, на котором команда принимает решение о дальнейшей стратегии тестирования.
4. **Gate-критерий** — условие остановки или продолжения тестового процесса.
5. **Pass/Fail** — явно зафиксированный итог входной проверки.